

WA-PRO-CRM — Especificación Técnica y Análisis de Arquitectura

Proyecto: WA-PRO-CRM (repositorio: `uniwai`)
Versión del documento: 1.0
Fecha: 2026-07-02
Estado: Borrador para aprobación — sin implementación hasta firma de este spec
Autor: Sesión de diseño Superpowers (brainstorming)

1. Resumen ejecutivo

WA-PRO-CRM es un **SaaS CRM omnicanal multi-tenant** especializado en WhatsApp, orientado a PYMEs latinoamericanas (referencia competitiva: [ChatPro](#)). Combina:

- Conexión **oficial** (Meta Cloud API) y **no oficial** (Baileys / QR vía WebSockets)
- **Constructor visual de bots** (React Flow) con live preview en mockup de smartphone
- **Motor anti-ban** y red P2P de calentamiento de números
- **CRM Kanban** con human takeover y reanudación de flujo
- **E-commerce in-chat** con catálogo, checkout, PostGIS para envíos locales
- **IA generativa BYOK** con middleware anti-prompt-injection
- **Facturación** por planes (límites de bots, vendedores, etc.) vía Mercado Pago
- **Marketing** (remarketing, listas Excel hasta 499 contactos)
- **Reportes** exportables a Excel (2 hojas: resumen órdenes + compradores)
- **Panel Superadmin** (suscripciones, estadísticas globales, baneos)

Decisión arquitectónica clave: Monorepo con separación clara entre UI (Next.js), API (Hono/Bun), workers (Baileys, BullMQ, warmup) y paquete compartido de base de datos (Prisma).

Advertencia legal/técnica: La conexión no oficial (Baileys) viola los Términos de Servicio de WhatsApp/Meta. El producto debe etiquetarlo explícitamente, ofrecer la API oficial como canal recomendado, y documentar riesgo de baneo permanente del número. Las funciones "anti-ban" reducen probabilidad pero **no garantizan** cumplimiento ni ausencia de baneos.

2. Análisis competitivo (ChatPro y diferenciadores)

2.1 Referencia: ChatPro ([chatpro.pe](#))

Aspecto	Observación
Posicionamiento	Plataforma SaaS de automatización WhatsApp para negocios en Perú/LATAM
Propuesta	Bots, CRM, campañas — enfoque en facilidad para PYME
Acceso	Plataforma web con login; onboarding orientado a negocio

2.2 Diferenciadores propuestos para WA-PRO-CRM

Funcionalidad	ChatPro (típico del segmento)	WA-PRO-CRM (objetivo)
---------------	-------------------------------	-----------------------

Bot Builder visual	Flujos predefinidos / editor básico	React Flow drag-and-drop + live preview sin guardar
Anti-ban	Delays básicos	Spintax, presencia composing, red P2P warmup, multitext
E-commerce	Catálogo / links externos	Google Sheets + catálogo WA + PostGIS tarifa envío local
IA	Respuestas automáticas	BYOK multi-proveedor + base conocimiento + anti-injection
Pagos in-chat	Variable	Mercado Pago nativo, credenciales en dashboard (no .env tenant)
Human takeover	Parcial en competidores	Toggle por prospecto + reanudación exacta vía currentNodeId en Redis
Marketing	Campañas masivas	Listas Excel (≤ 499), remarketing inteligente
Reportes	Básicos	Excel 2 hojas (órdenes + compradores) con estadísticas
Superadmin	—	Panel maestro, suspensión/ban, stats globales, keys IA compartidas

3. Stack tecnológico obligatorio

Capa	Tecnología	Justificación
Runtime	Bun 1.x	WebSockets nativos, bajo overhead, compatible con workers
API	Hono (recomendado sobre Elysia por ecosistema middleware)	Ligero, tipado, edge-ready, middleware JWT/CORS
Frontend	Next.js 15 App Router, React 19, TypeScript	SSR landing, RSC para dashboard, SEO
UI	Tailwind CSS v4 , shadcn/ui , Framer Motion	Design system consistente; motion con prefers-reduced-motion
Estado cliente	Zustand	Bot builder + live preview reactivo
Lienzo bots	React Flow	Nodos/edges, serialización JSON
ORM / DB	Prisma + PostgreSQL (Docker local dev)	Tipado, migraciones, extensiones
Extensiones PG	pgvector (embeddings RAG), PostGIS (distancia envíos)	IA + logística espacial
Colas	Redis + BullMQ	Warmup, envío async, webhooks, IA
WhatsApp oficial	Meta Cloud API	Canal compliant
WhatsApp no oficial	@whiskeysockets/baileys	QR, WebSocket; worker aislado

Storage archivos	S3-compatible (MinIO dev, AWS/R2 prod)	Imágenes/PDF en flujos
Pagos	Mercado Pago	Credenciales por tenant en DB cifradas

3.1 ¿Se puede hacer landing + CRM con Bun?

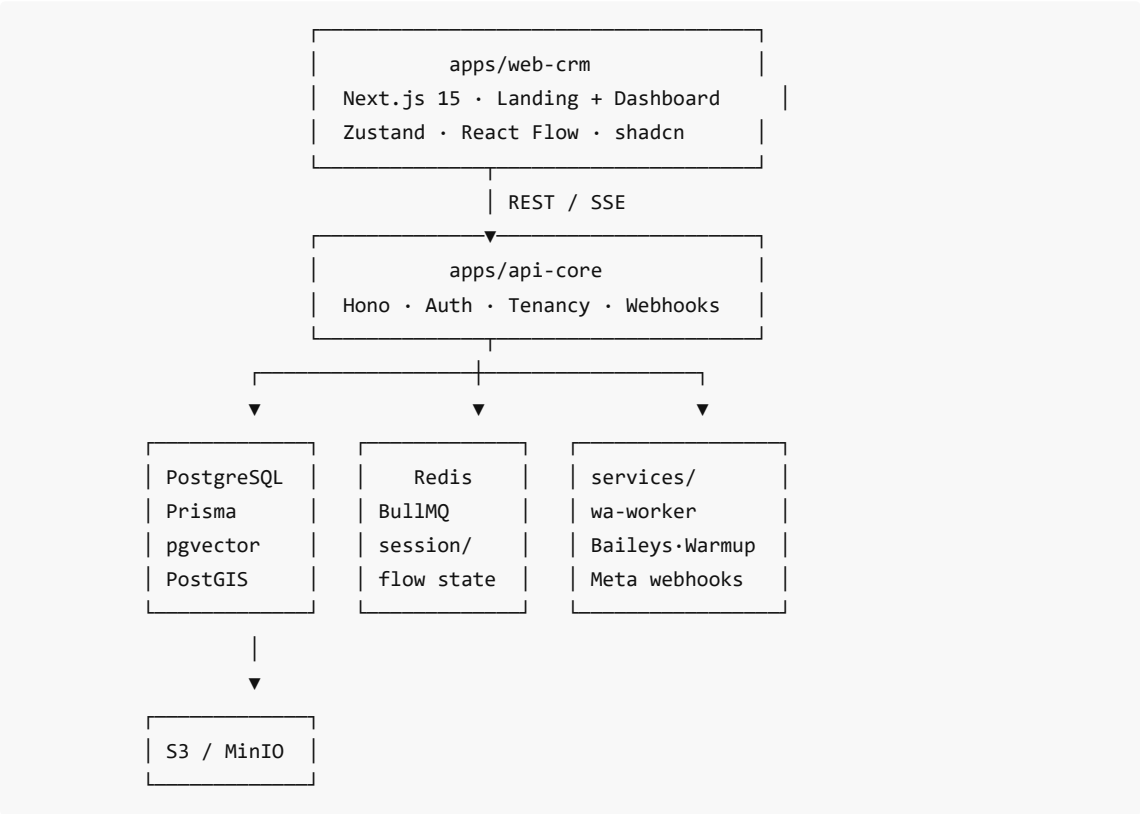
Sí, con arquitectura híbrida recomendada:

```
apps/web-crm      → Next.js (build Node/Bun; dev con bun o next)
apps/api-core     → Hono servido con Bun
services/wa-worker → Bun puro (Baileys + BullMQ consumers)
```

- **Landing y CRM:** Next.js 15 (no reemplazar por Hono SSR para el dashboard complejo).
- **API y workers:** Bun como runtime prioritario.
- **Un solo `bun install`** en monorepo (workspaces) unifica dependencias.

4. Arquitectura del sistema

4.1 Diagrama de alto nivel



4.2 Estructura monorepo

```
uniwai/
├── apps/
│   └── web-crm/           # Next.js 15 – landing, dashboard, bot builder
```

```
|   └─ api-core/           # Hono API – REST, webhooks Meta, auth
└─ packages/
|   └─ database/           # Prisma schema, client, migrations
|   └─ shared/             # Tipos, validaciones Zod, constantes
|   └─ ui/                 # (opcional) componentes shadcn compartidos
└─ services/
|   └─ wa-worker/          # Bun – Baileys, warmup P2P, flow executor
└─ docker/
|   └─ docker-compose.yml  # postgres+pgvector+postgis, redis, minio
|   └─ init-extensions.sql
└─ docs/
|   └─ superpowers/specs/
└─ package.json            # bun workspaces
```

4.3 Multi-tenancy estricto

- Toda tabla de negocio incluye `tenantId` (UUID).
- **Prisma middleware** inyecta `tenantId` desde JWT en cada `find/create/update/delete` .
- **Imposible** omitir filtro: queries sin `tenantId` fallan en middleware (fail-closed).
- Superadmin usa `role=SUPERADMIN` + contexto sin tenant para panel global (rutas separadas `/admin/*`).

4.4 Roles y permisos (RBAC)

Rol	Alcance	Permisos clave
SUPERADMIN	Plataforma	Planes, precios, suspender/banear tenants, stats globales, pool keys IA opcionales
OWNER	Su tenant	Bots, flujos, MP creds, equipo, IA BYOK, warmup config, reportes
VENDEDOR	Su tenant (limitado)	Kanban, chat, toggle bot por prospecto — sin configuración ni billing

Matriz guardada en `packages/shared/src/rbac.ts` ; enforced en Hono middleware + UI route guards.

5. Módulos funcionales (detalle)

5.1 Constructor de Bots (Bot Builder) + Live Preview

UI: Pantalla dividida — lienzo React Flow (70%) + `SmartphoneMockup` (30%).

Nodos MVP:

Nodo	Comportamiento
<code>MessageMultiText</code>	Array de variantes + Spintax {hola buenos días}
<code>Buttons</code>	Hasta 3 botones reply (límites WA)
<code>List</code>	Lista de selección WhatsApp
<code>Media</code>	Imagen/PDF → upload S3 → URL en mensaje
<code>Condition</code>	Rama por texto/regex/variable

SetVariable	Guardar en contexto prospecto (Redis + DB)
AIResponse	Llama middleware IA con businessContext
ProductCatalog	Lee Sheets o catálogo WA nativo
Checkout	Carrito, total, ubicación, envío, pago
HumanHandoff	Pausa bot, notifica vendedor
Delay	Espera N segundos (anti-ban)

Live Preview: Zustand store `useBotPreviewStore` sincronizado con nodo seleccionado en React Flow. Renderiza burbujas WA sin persistir. Avanzar/retroceder con iconos en toolbar del mockup.

Persistencia: `BotFlow.nodes` y `BotFlow.edges` como JSONB en Prisma. Hasta **20 flujos** por tenant (límite configurable por plan).

Plantillas: `FlowTemplate` globales (superadmin) + por tenant; clonar al crear flujo.

5.2 Motor Anti-Ban y Calentador P2P

Worker: `services/wa-worker/src/jobs/warmup-p2p.ts` (BullMQ queue `warmup`).

Lógica:

1. Seleccionar instancias `WhatsAppInstance` con `isInWarmupNetwork = true`.
2. Emparejar pares (grafo no dirigido, evitar mismo tenant si política lo exige).
3. Antes de enviar: evento Baileys `presenceSubscribe` + `sendPresenceUpdate('composing')`.
4. Duración composing: `15-45 ms × caracteres` (aleatorio por carácter).
5. Aplicar Spintax al texto desde `WarmupMessageTemplate` (multitext del tenant).
6. Registrar en `WarmupLog` (origen, destino, timestamp, texto hash).

Config por número (OWNER):

- Lista de números destino manual
- Opt-in a red CRM (`joinWarmupNetwork`)
- Multitext de mensajes de calentamiento
- Horarios y límites diarios

5.3 E-commerce In-App y Logística PostGIS

Fuentes de productos:

1. API Google Sheets (OAuth o service account por tenant)
2. Catálogo nativo WhatsApp Business (Meta Commerce API)

Flujo checkout:

1. Usuario selecciona productos en nodo catálogo
2. Bot construye `Order` draft en Redis + DB
3. Solicita ubicación (pin WA) o dirección texto
4. Si envío local: `ST_DistanceSphere(tenant.location, user.location)` → tarifa por km/zona
5. Si agencia: tarifa fija configurada por tenant
6. Pago: Mercado Pago link/in-chat o contraentrega (flag `cashOnDelivery`)
7. Confirmación → alerta OWNER + registro en Kanban columna "Cierre"

Modelo PostGIS: `Tenant.storeLocation` y `Order.deliveryLocation` como `geography(Point, 4326)` .

5.4 CRM Kanban y Human Takeover

Columnas default: Lead → Contactado → Negociación → Cierre (customizables por tenant).

Tarjeta prospecto: nombre, teléfono, último mensaje, etiquetas, bot on/off.

Human takeover:

- `Contact.botEnabled = false` → mensajes van al inbox del vendedor
- Al reactivar: worker lee Redis `flow:contact:{id}:currentNodeId` y ejecuta desde ese nodo
- Estado flujo también en `Contact.flowState` (JSONB backup)

Chat embebido: Panel lateral al seleccionar tarjeta; mismo componente que inbox.

5.5 Middleware IA Segura (BYOK + Anti-Injection)

Proveedores: OpenAI, Gemini, DeepSeek (adapter pattern).

Config:

- SUPERADMIN: keys pool opcionales para plan básico
- OWNER: BYOK en dashboard (cifradas AES-256-GCM con `ENCRYPTION_KEY` servidor)

Pipeline:

```
userMessage → sanitizer → injectionDetector → contextBuilder(businessContext + RAG pgvector)
               → LLM → outputValidator → send or reject
```

Rechazos explícitos: clima, tecnología del CRM, código, system prompts, temas fuera de `businessContext` .

RAG: `KnowledgeDocument` chunked + embeddings en pgvector; búsqueda top-k por tenant.

5.6 Planes, facturación y Mercado Pago

Planes (ejemplo):

Plan	Bots	Vendedores	Flujos	Marketing	Precio
Básico	1	0	5	No	S/ XX
Pro	3	3	15	Sí (499)	S/ XX
Enterprise	10	ilimitado	20	Sí	Custom

Mercado Pago: `TenantPaymentConfig` (`access_token`, `public_key`) cifrados; webhook en api-core.

Límites: middleware verifica `Subscription.limits` antes de crear recurso.

5.7 Marketing y remarketing

- Import Excel ≤499 filas: `phone`, `name`, `category`, `tag`
- Campañas con plantilla aprobada (Meta) o sesión Baileys (riesgo)
- Remarketing: reglas sobre `Contact` sin respuesta / abandono carrito
- Cola BullMQ `campaigns` con rate limit estricto

5.8 Reportes y exportación Excel

Endpoint: GET /api/reports/sales/export?from=&to=

Libro Excel (2 hojas):

1. **Resumen Órdenes:** totales, ticket medio, por estado, por día
2. **Compradores:** contactos con órdenes, monto, fecha, productos

Librería: exceljs en api-core.

5.9 Superadmin y alertas

- Dashboard: MRR, tenants activos, mensajes/día, instancias baneadas
- Acciones: suspender tenant, banear, resetear instancia WA
- Alertas OWNER: compra realizada, prospecto hot, error instancia WA (email + in-app)

5.10 Landing

- Next.js marketing en / (pública)
- App en /app/* (auth required)
- SEO: meta, schema Organization, CTA planes

6. Modelo de datos (Prisma — resumen)

6.1 Entidades principales

```
Tenant └─ User (role)
      │─ Subscription / Plan
      │─ WhatsAppInstance (official|baileys, warmup flags)
      │─ BotFlow (nodes Json, edges Json)
      │─ FlowTemplate
      │─ Contact (botEnabled, flowState, kanbanColumn)
      │─ ChatMessage
      │─ Product (source: sheets|wa_catalog)
      │─ Order (delivery geography, totals, paymentStatus)
      │─ KnowledgeDocument + Embedding (pgvector)
      │─ TenantPaymentConfig (encrypted)
      │─ WarmupConfig / WarmupLog
      │─ Campaign / CampaignContact
      └─ AuditLog
```

6.2 Campos críticos

WhatsAppInstance:

- `connectionType` : META_CLOUD | BAILEYS
- `isInWarmupNetwork` : boolean
- `warmupPartnerIds` : string[]
- `status` : CONNECTED | DISCONNECTED | BANNED | QR_PENDING

Contact:

- `currentNodeId` : string (sync Redis)

- botEnabled : boolean
- kanbanColumnId : FK

BotFlow:

- nodes : Json (React Flow)
- edges : Json
- version : int (optimistic locking)

7. Seguridad (obligatorio)

Control	Implementación
Tenancy	Prisma middleware + tests
Auth	JWT corta vida + refresh httpOnly; validar iss/aud/exp
CORS	Allowlist por entorno
Headers	HSTS, nosniff, DENY frame, Referrer-Policy
Rate limit	5–10/min auth; 60–120/min API; colas WA con throttle
Secrets	MP keys, IA keys cifradas; nunca en cliente
CSP	Next.js headers en producción
Source maps	Deshabilitados en prod
Prompt injection	Middleware dedicado + tests adversariales
Baileys	Worker aislado; sin exposición de sesión al frontend

8. Roadmap por fases (descomposición)

El prompt original pide monorepo → schema → worker warmup. **Recomendación:** ejecutar warmup (Fase 4) solo después de tener instancias WA estables (Fase 2).

Fase	Entregable	Duración estimada
F0	Monorepo, Docker, Prisma base, auth multi-tenant	1–2 sem
F1	Meta Cloud API + inbox básico	2 sem
F2	Baileys worker + QR connect	2 sem
F3	Bot Builder + preview + executor	3 sem
F4	Kanban + human takeover	2 sem
F5	Warmup P2P + anti-ban	2 sem
F6	IA BYOK + RAG + anti-injection	2 sem
F7	E-commerce + PostGIS + MP	3 sem

F8	Planes, billing, superadmin	2 sem
F9	Marketing + reportes Excel	2 sem
F10	Landing polish + hardening	1 sem

MVP comercial mínimo: F0 + F1 + F3 (solo Meta) + F4 + F6 (básico).

9. Orden de implementación (ajustado)

El usuario solicitó:

1. Estructura monorepo
2. `schema.prisma` completo
3. Worker calentador P2P

Orden aprobado para el plan de implementación (con ajuste justificado):

1. Monorepo + Docker + Prisma schema **completo** (todas las entidades, migración inicial)
2. api-core: auth, tenancy middleware, health
3. web-crm: shell Next.js, login, layout dashboard
4. wa-worker: conexión Meta + Baileys (antes que warmup)
5. Bot builder + executor
6. Kanban + takeover
7. **Warmup P2P worker** (depende de 4)
8. Resto de módulos por fases

10. Entorno de desarrollo — Base de datos

Recomendación para DEV: PostgreSQL local vía Docker (no Supabase cloud para dev).

```
# docker-compose.yml (resumen)
services:
  postgres:
    image: pgvector/pgvector:pg16
    # + postgis via init script
  redis:
    image: redis:7-alpine
  minio:
    image: minio/minio
```

Extensiones: `CREATE EXTENSION vector;` `CREATE EXTENSION postgis;`

Prisma: `binaryTargets` para Bun; `previewFeatures` si se usa `postgresqlExtensions`.

Producción: RDS/Neon/Supabase Postgres con mismas extensiones; Redis managed (Upstash/ElastiCache).

11. UI/UX (principios ui-ux-pro-max)

- **Product type:** SaaS B2B dashboard + landing
- **Estilo:** Clean minimal + dark mode opcional; shadcn defaults customizados

- **Kanban:** columnas scroll horizontal mobile; tarjetas 44px touch targets
- **Bot builder:** undo/redo, zoom React Flow, minimap
- **Motion:** Framer $\leq 300\text{ms}$; `LazyMotion` + `m`; reduced-motion
- **Accesibilidad:** contraste 4.5:1, focus rings, labels en forms
- **Preview smartphone:** frame fijo 390×844; no depender de hover

12. Riesgos y mitigaciones

Riesgo	Impacto	Mitigación
Baneo Baileys	Alto	API oficial como default; warnings UI; warmup no garantiza nada
Meta policy cambios	Alto	Adapter pattern; feature flags
Complejidad scope	Alto	Fases estrictas; MVP sin marketing ni e-commerce primero
Prompt injection	Medio	Middleware + tests + sin tools al LLM
Rate limits WA	Medio	BullMQ throttling por instancia
MP creds en DB	Medio	Cifrado + audit log accesos

13. Criterios de aceptación — Fase 0 (fundación)

- ☐ `bun install` en raíz instala todos los workspaces
- ☐ `docker compose up` levanta PG+Redis+MinIO con extensiones
- ☐ `prisma migrate dev` aplica schema sin errores
- ☐ `api-core` responde `/health` con Bun
- ☐ `web-crm` arranca en dev con login mock o real
- ☐ Test: usuario tenant A no ve datos tenant B

14. Próximos pasos (post-aprobación)

1. Usuario aprueba este spec
2. Invocar skill **writing-plans** → plan de implementación por tareas
3. Ejecutar Fase 0: monorepo + schema + Docker
4. Commits incrementales por módulo

15. Glosario

Término	Definición
BYOK	Bring Your Own Key — cliente usa su API key de IA
Spintax	<code>{opción1 opción2}</code> — variación de texto
Human takeover	Vendedor toma chat; bot pausado
Warmup network	Red P2P de números que se mensajan entre sí

Tenant	Instancia aislada de un cliente del SaaS
--------	--

Documento generado en flujo Superpowers brainstorming. Pendiente de revisión y aprobación del usuario antes de escribir código.